

18/04/2024

1

a) (1 punto)

¿En que dirección de memoria se encuentra la pila justo al empezar la ejecución de la rutina 'main' de sistema?

Es la dirección `0x3ff4`

¿Como la has encontrado?

```
$ make gdb
(gdb) b main          # breakpoint system main
(gdb) print $esp      # print stack addr
```

¿Corresponde a algún proceso del vector task?

No corresponde a ningún proceso del vector task, ya que todavía no se han ejecutado `set_eflags` ni `set_seg_regs` para cambiar `esp` a `&task[4]`

¿Cómo lo has hallado?

Continuando desde la ejecución anterior:

```
(gdb) b printk
(gdb) print $esp
$5 = (void *) 0x18fd8 <task+16344>
(gdb) print &task[4]
$6 = (union task_union *) 0x19000 <task+16384>
```

También, leyendo el código, vemos como se pasa el argumento `&task[4]` a la función `set_seg_regs`.

b) (1 punto) En el código inicial os damos el tratamiento de las excepciones ya implementado.

¿Es posible ver el código ensamblador de la rutina de tratamiento de la excepción de división por zero?

Sí.

Caso afirmativo muestra el código y cómo lo has hallado.

```
$ objdump -d libzeos.a | less
:/divide

00000210 <divide_error_routine>:
 210:  55                      push    %ebp
 211:  89 e5                   mov     %esp,%ebp
 213:  83 ec 14                sub     $0x14,%esp
 216:  68 00 00 00 00          push    $0x0
 21b:  e8 fc ff ff ff         call    21c <divide_error_routine+0xc>
 220:  c7 04 24 00 00 00 00    movl    $0x0,(%esp)
 227:  e8 fc ff ff ff         call    228 <divide_error_routine+0x18>
 22c:  83 c4 10                add     $0x10,%esp
 22f:  eb fe                  jmp     22f <divide_error_routine+0x1f>
 231:  8d b4 26 00 00 00 00    lea     0x0(%esi,%eiz,1),%esi
 238:  8d b4 26 00 00 00 00    lea     0x0(%esi,%eiz,1),%esi
 23f:  90                      nop
```

c) (1 punto) Dado un fichero exam.o que contiene la función shared implementada

¿Como debe ser la línea (o líneas) de comandos para linkar este objeto y que puedas usar la función tanto desde el código de sistema como de usuario?

Añadir al Makefile:

```
SYSOBJ = ... exam.o
USROBJ = exam.o
```

2

El bucle de copia de datos de usuario implica que hay que modificar entradas de la tabla de páginas (TP) del proceso actual temporalmente para mapear la zona de datos del proceso hijo. Para ahorrarnos esta modificación y dado que sólo estamos usando una entrada del directorio, queremos mapear temporalmente toda la TP del proceso hijo en la entrada 1 del directorio, y así realizar la copia de toda la zona de datos con una única llamada a copy_data. Implementa esta modificación del sys_fork.

No tinc clar com fer-ho, dona la sensacio que a ZeOS nomes es pot allocar 1 directori per proces.

3

Queremos añadir a nuestro ZeOS una funcionalidad para leer 1 tecla del teclado :

```
int read(char* b);
```

Esta llamada bloquea al proceso actual en una lista de bloqueados en el teclado hasta que se pulse una tecla, momento en que desbloqueará al proceso y lo pondrá en ejecución, copiando la tecla leída al buffer 'b'. Si varios procesos usan esta llamada, el orden de desbloqueo tiene que seguir un orden FIFO. Esta llamada debe devolver error si el buffer no se encuentra dentro del espacio de direcciones del proceso.

Esta llamada a sistema tiene que usar la interrupción 130 para realizar la entrada a sistema (en lugar de los mecanismos ya implementados) y debe ejecutar la rutina de servicio directamente pues será el único servicio accesible mediante esta interrupción.

El parámetro se pasará por registro.

Implementa las funciones de sistema siguientes para gestionar la lista de bloqueados en el teclado:

- `void block_for_keyboard(void)` : bloquea el proceso actual.
- `void unblock_first()` : desbloquea y pasa a ejecutar el primer proceso de la lista. Si no hay procesos bloqueados esta función no hace nada.

La solución tiene que ser genérica y funcionar de forma eficiente para cualquier número de procesos.

a) (0,75 puntos) Implementa el código del wrapper de la llamada read.

`libc_sys.S` (where wrappers are)

```
ENTRY(read)
    pushl %ebp                # dynamic link
    movl %esp, %ebp

                                # no need to save edx
    movl 8(%ebp), %edx        # edx = char* b
    int $130                  # SYSCALL custom for interrupt 130

    cmpl $0, %eax             # check if error
    jge read_err_fi
    pushl %eax                # pass error value to errno
    call set_errno            # set errno to provided value
    addl $4, %esp             # remove parameter from stack once call finished
    movl $-1, %eax            # set return value to -1 as specified
read_err_fi:
    popl %ebp                # restore registers
    ret
```

`libc.h`

```
int read(char* b);
```

b) (0,75 puntos) Implementa el código del handler de esta llamada a sistema.

`entry.S`

```
ENTRY(read_system_call_handler)
    SAVE_ALL                # Save the current context
    call sys_read           # Call service routine in sys.c
    movl %eax, 0x18(%esp)   # Change the EAX value in the stack
    RESTORE_ALL             # Restore the context
    iret
```

c) (0,5 puntos) Indica qué estructuras de datos se tienen que añadir y/o modificar. Añade el código necesario para inicializarlas.

- Tenemos que añadir una cola para los bloqueados del teclado.

keyboard.h

```
extern struct list_head keyboard_blocked;
```

keyboard.c

```
struct list_head keyboard_blocked;
```

interrupt.c::setIdt()

```
set_handlers();
INIT_LIST_HEAD(&keyboard_blocked); // new
```

d) (0.5 puntos) Implementa la rutina block_for_keyboard.

keyboard.h

```
void block_for_keyboard();
```

keyboard.c

```
void block_for_keyboard() {
    struct task_struct* task = current();
    // task->state = ST_BLOCKED; // Only needed if update_process_state_rr
    // does not set it

    update_process_state_rr(task, &keyboard_blocked);
}
```

```
    sched_next_rr();  
}
```

e) (0.5 puntos) Implementa la rutina `unblock_first`.

`keyboard.h`

```
void unblock_first();
```

`keyboard.c`

```
void unblock_first() {  
    if (list_empty(&keyboard_blocked)) {  
        return;  
    }  
    struct list_head* head = list_first(&keyboard_blocked);  
    list_del(head);  
    struct task_struct* task = list_head_to_task_struct(head);  
    task->state = ST_RUN;  
    list_add(head, &readyqueue); // Add to the first entry on the list  
    sched_next_rr();             // force a task_switch  
}
```

f) (1 punto) Implementa el código de la rutina `sys_read`.

`sys.c`

```
int sys_read(char* b) {  
    // Check if buffer is in address space of process  
    if (!access_ok(LECTURA, b, 1)) return -EFAULT;  
  
    block_for_keyboard();  
    *b = char_read;  
    return 0;  
}
```

g) (1 punto) ¿Es necesario modificar alguna otra llamada a sistema o parte del sistema para implementar por completo esta funcionalidad?

Si es así implementa los cambios necesarios.

- Sí, tenemos que modificar la `keyboard_routine` para añadir el código necesario para desbloquear el proceso cuando se pulsa una tecla.
- Y para guardar esa tecla en una variable que `sys_read` pueda leer.

keyboard.h

```
extern char char_read;
```

keyboard.c::keyboard_routine

```
char char_read;
void keyboard_routine() {
    Byte event = inb(0x60);
    // make/break
    // make: key pressed
    // break: key released
    Byte make = !(event >> 7);
    Byte code = event & 0x7f;
    if (!make) return;

    char_read = char_map[code];
    unblock_first(); // unblock first keyboard_blocked
}
```

- También se tiene que añadir el `read_system_call_handler` como interrupt handler

entry.h

```
void read_system_call_handler();
```

interrupt.c::setIdt

```
setInterruptHandler(33, keyboard_handler, 0);
setInterruptHandler(130, read_system_call_handler, 3); // new
```